UN MANUALE DI BM

RIFF: scrivere musica per bm

Scrivi ritmi e melodie in poche righe, ascoltali subito e usali nel tuo gioco.

Indice

Che cos'è RIFF **4**

Che cosa fa 4

Perché conta 4

Da sapere 4

La tua prima canzone in 5 minuti **5**

La notazione per intero **7**

La mini-notazione 7

Parole, note e numeri 7

Costruire i pattern 8

Modificare i pattern 8

Scale e segnali 9

Strumenti ed effetti **10**

Scegliere lo strumento 10

Il suono di ogni nota 10

Il kit 11

Pattern, canzoni e struttura **12**

Più parti insieme 12

Strofa e ritornello 12

Ripetere e cambiare 12

Tempo 12

Trasformare un pattern in una canzone del bank 12

Usare una canzone RIFF in un gioco **13**

Le tre chiamate che servono a ogni gioco 13

Che cosa fa un frame lento 13

Le funzioni per il gioco 13

Codice dal vivo dal gioco 13

Voci, effetti sonori e canzoni del bank 14

Esempi **15**

Un ritmo, un basso e le tastiere 15
Un pad, un arpeggio e la batteria 15
Una melodia su una scala 15
Il kit e gli effetti 15
Righe singole per la riga di F7 15
Fermare e far partire dal codice 16

Errori comuni 17

Punti aperti 18

LEGGIMI

Che cos'è. Il manuale di RIFF, il linguaggio di bm per scrivere musica in poche righe di Lua. Il contenuto viene da `docs/RIFF.md`, `docs/api/riff.md` e `tests/riff/cart.lua`.

Come usarlo. Fai la prima canzone (capitolo 2) passo per passo, poi tieni a portata di mano i capitoli sulla notazione. Tutto ciò che sta in un riquadro con bordo e ombra va copiato così com'è.

Che cosa manca. Ogni lacuna è segnata con un'etichetta evidenziata DA COMPLETARE ed elencata nell'ultimo capitolo; niente è inventato.

Che cos'è RIFF

RIFF è un modo di scrivere la musica come testo. Un ritmo o una melodia è una stringa breve; funzioni concatenate la modificano. Lo stile viene da TidalCycles e Strudel.

Che cosa fa

- Scrivi `s "kick*4, ~ snare, hat*8"` e ottieni un ritmo di batteria.
- Lo cambi con i metodi: `:fast(2)`, `:gain(.9)`, `:room(.4)`.
- Suona con il sintetizzatore e gli strumenti di bm, e con i suoni del **bank** della cartuccia, per nome.

Perché conta

Una canzone per un gioco sono poche righe, non un file di note. La scrivi e la senti dal vivo in **bm Code**, la trasformi in una canzone del bank in **bm Sound**, oppure la suoni dal tuo gioco mentre gira.

Da sapere

- RIFF è una libreria del kernel (`src/script/riff.lua`, come bmlib). La carichi con `require "riff"`. Funziona nei file `.bm`, sul Pi e sull'RGB30, e in bmhost sul PC.
- Il tempo si conta in **cicli**. Un ciclo è una battuta di quattro movimenti: 2 secondi a 120 BPM (`setcps(0.5)`, il valore iniziale).
- Un pattern dice quali eventi cadono in un intervallo di cicli. A ogni frame, `R.update()` mette in coda le note dei momenti successivi (circa 0,2 s avanti) con `play_at()`. L'interrupt del suono le avvia in tempo, qualunque sia la frequenza dei frame del gioco.
- Le note aspettano in una coda di al massimo 160. RIFF non prende mai le voci di una canzone del bank né di un effetto sonoro.

La tua prima canzone in 5 minuti

Scriverai un ritmo, lo ascolterai, aggiungerai un basso e li sentirai insieme. Ti serve bm Sound aperto su un bank (la riga di F7 suona con i suoni del bank aperto e con gli strumenti).

> **PROVA**
> Segui i passi nell'ordine. Dopo ogni passo, guarda che cosa deve mostrare lo schermo.

1. Apri **bm Sound**. **[DA COMPLETARE: come avviare bm Sound dal menu della console]**
2. Premi **F7** (oppure scegli *Riff...* nel menu). Si apre una riga dove scrivi un pattern, con un primo esempio già dentro. Con i tasti su e giù scorri otto esempi.
3. Cancella l'esempio e scrivi questa riga:

```
s "kick*4, ~ snare, hat*8"
```

È una battuta di quattro movimenti: quattro kick, uno snare nella seconda metà, otto hat.

4. Premi **Enter**. Senti il ritmo e la riga di stato dice `playing`. Le parole delle note che suonano si illuminano.
5. Rendilo più morbido e aggiungi una stanza. Cambia la riga in:

```
s "kick*4, ~ snare, hat*8" :gain(.9) :room(.2)
```

Premi **Enter** di nuovo: il ritmo suona con la modifica.

6. Prova una melodia. Scrivi questo e premi **Enter**:

```
note "c4 e4 g4 c5" :s "epiano" :room(.4)
```

Senti quattro note di un accordo di Do maggiore con il piano elettrico, una per movimento.

7. Premi **Tab** per scegliere quante battute tenere (1, 2, 4, 8 o 16), poi **Ctrl+Enter**. Il pattern entra nel bank come **nuova canzone**. La riga di stato dice `riff: N bars into song M`. **Ctrl+Z** lo annulla.
8. Premi **Esc** per chiudere la riga e fermare la musica.

Ora suona lo stesso codice da bm Code.

1. Apri una scheda di bm Code e scrivi queste righe. **[DA COMPLETARE: come aprire bm Code e una scheda nuova]**

```
setcpm(30)
drums = s "kick*4, ~ snare, hat*8" :gain(.9)
bass  = note "<c2 a1 f1 g1>" :s "acid" :lpf(sine:range(300, 1800):slow(4))
```

2. Premi **Ctrl+Enter**. In una scheda che contiene solo pattern suona tutta la scheda. La riga di stato dice `MUSIC 2`: due pattern stanno suonando. Le parole delle note che suonano si illuminano.
3. Cambia `kick*4` in `kick*2` e premi di nuovo **Ctrl+Enter**. La musica cambia senza pause.
4. Premi **Ctrl+.** per fermare tutto.

Se sbagli, la riga viene segnata, la riga di stato dice che cosa non va e la musica che suonava continua. Correggi la riga e premi di nuovo **Ctrl+Enter**.

[DA COMPLETARE: shots/riff-sound-line.png]

DA COPIARE

```
s "kick*4, ~ snare, hat*8" :gain(.9) :room(.2)
note "c4 e4 g4 c5" :s "epiano" :room(.4)
```

La notazione per intero

Una stringa è **un ciclo** diviso in parti uguali. Questa è la mini-notazione.

La mini-notazione

Scritto	Che cosa fa	Esempio
`a b c d`	una sequenza: quattro parti del ciclo	`"kick hat snare hat"`
`~ o -`	una pausa	`"kick ~ kick ~"`
`[a b]`	un gruppo: due parti nello spazio di una	`"kick [hat hat]"`
`a, b`	insieme (anche dentro `[ ]`)	`"kick*4, ~ snare"`, `"[c,e,g]"`
`<a b c>`	uno per ciclo, a turno	`"<c2 a1 f1 g1>"`
`a*3`	tre volte più veloce (anche `a*<2 4>`)	`"hat*8"`
`a/2`	due volte più lento	`"pad/2"`
`a!3`, `a !`	ripetuto (tre volte, una volta in più)	`"kick!3 snare"`
`a@3`, `a _ _`	più lungo: pesa tre parti	`"c@3 e"`, `"c _ _ e"`
`a?`, `a?0.3`	suona a caso, il 50% (o il 70%) delle volte	`"hat*16?"`
`a(3,8)`, `a(3,8,2)`	ritmo euclideo: 3 colpi distribuiti su 8 passi (ruotato di 2)	`"kick(3,8)"`
`{a b c}%4`	polimetro: 4 passi per ciclo dalla sequenza di 3	`"{kick hat snare}%4"`
`a | b`	uno dei due a caso, a ogni ciclo	`"[snare | clap]"`
`a . b c`	gruppi separati da un punto (`[a] [b c]`)	`"kick . hat hat hat"`
`kick:2`	una variante: due semitoni sopra (per la batteria)	`"kick:2 kick"`
`kit:2`	un colpo del kit della console (campioni): da 0 `bd` a 7 `cb`, alla sua velocità	`"kit:0 kit:2"`, `"bd sd hh"`

Parole, note e numeri

Le parole sono note (`c`, `eb4`, `f#2`), numeri (`0 2 -1 0.5`) o nomi di strumenti.

- Senza ottava, una nota è nell'ottava 3: `c3` è 48.
- `c4` è 60, il Do centrale, come in Strudel.
- Un errore in una stringa dice dove: `riff: missing ] at 6 in "a [b c"`.

Costruire i pattern

Funzione	Che cosa fa
`s(p)`, `sound(p)`	strumenti: un nome del bank della cartuccia, un preset (`"kick"`, `"epiano"`, `instruments()`), un numero (il suono del bank)
`note(p)`	note: nomi o numeri MIDI
`n(p)`	gradi di una scala (`:scale`); senza scala, Do maggiore da C3
`chord(p)`	accordi: `"<C Am F G7>"`, `"Fmaj7"`, `"Dm7"`, `"Bb"`, `"F#m"`, `"Csus4"`, `"Edim"`, in una disposizione vicina al Do centrale
`seq(a, b, ...)`, `cat(a, b, ...)`, `stack(a, b, ...)`	in sequenza dentro il ciclo, un ciclo ciascuno, insieme
`timecat({3, a}, {1, b})`	in sequenza con pesi
`arrange({4, a}, {4, b})`	sezioni di più cicli, una dopo l'altra
`run(n)`	`0 1 2 ... n-1`
`pure(v)`, `silence`, `mini(s)`	un valore per ciclo, niente, una stringa di mini-notazione
`piece(t)`	un brano di `ai.music` (o di `R.bake`) come pattern: un ciclo ogni 16 passi

Un argomento che è una stringa è sempre mini-notazione, quindi quasi tutto può essere un pattern: `:lpf("<400 2000>")`, `:fast("<1 2>")`, `:scale("<C:major A:minor>")`.

Modificare i pattern

I metodi si concatenano con i due punti (in Lua `p:fast(2)`). Con una stringa le parentesi si possono omettere: `note "c e g" :s "pluck"`.

Metodo	Che cosa fa
`:fast(n)`, `:slow(n)`	più veloce, più lento
`:early(t)`, `:late(t)`	prima, dopo (in cicli)
`:rev()`, `:palindrome()`	al contrario, un ciclo sì e uno no
`:ply(n)`	ogni evento ripetuto n volte
`:iter(n)`, `:iterBack(n)`	ogni ciclo parte un n-esimo più avanti
`:every(n, f)`, `:lastOf(n, f)`	f sul primo (o sull'ultimo) di ogni n cicli
`:sometimes(f)`, `:often(f)`, `:rarely(f)`, `:almostNever(f)`, `:almostAlways(f)`, `:sometimesBy(x, f)`, `:someCycles(f)`	f su una parte degli eventi (o dei cicli) a caso
`:degrade()`, `:degradeBy(x)`	toglie a caso la metà (o una parte x) degli eventi

Metodo	Che cosa fa
`:euclid(k, n, [rot])`, `:struct("x ~ x x")`, `:mask("1 0 1 1")`	la struttura di un ritmo, o solo gli eventi dove la maschera è vera
`:off(t, f)`, `:superimpose(f)`, `:layer(f, g, ...)`	insieme a una copia modificata (spostata di t)
`:jux(f)`, `:juxBy(w, f)`	il pattern a sinistra, f del pattern a destra
`:chunk(n, f)`	f su un n-esimo del ciclo, uno diverso ogni volta
`:linger(x)`	ripete la prima parte x del ciclo
`:inside(n, f)`, `:outside(n, f)`	f su un pezzo di 1/n (o n) cicli
`:swing(n)`, `:swingBy(x, n)`	i passi pari di ogni 1/n un po' in ritardo
`:segment(n)`	n eventi per ciclo da un segnale
`:range(a, b)`, `:rangex(a, b)`	da 0..1 ad a..b (esponenziale)
`:add(x)`, `:sub(x)`, `:mul(x)`, `:div(x)`, `+`, `-`, `*`	sui numeri, sulla nota (o su `n`)
`:transpose(x)`, `:scale("C:minor")`	semitoni; i gradi `n` su una scala
`:arp("up")`	le note di un accordo una dopo l'altra: `up`, `down`, `updown`, `downup`, `converge`, `diverge`, `thumbup`
`:fmap(f)`	una funzione sui valori

Le funzioni da passare a `every`, `sometimes`, `off` sono funzioni Lua (`function(p) return p:fast(2) end`). Ci sono anche scorciatoie: `rev`, `fast(2)`, `slow(2)`, `ply(2)`, `late(0.25)`, `add(12)`, `degrade`. Così puoi scrivere `:every(4, fast(2))` o `:sometimes(add(12))`.

Scale e segnali

- **Scale**: `"C:major"`, `"A:minor"`, `"D4:dorian"`, e `phrygian`, `lydian`, `mixolydian`, `locrian`, `harmonic`, `melodic`, `pentatonic`, `minor_pentatonic`, `blues`, `chromatic`, `wholetone`. La tonica è nell'ottava 3 se non dai un'ottava.
- **Segnali** (0..1 su ogni ciclo, continui; usati come valori o con `:segment`): `sine`, `cosine`, `saw`, `isaw`, `tri`, `square`, `rand`, `perlin`, `irand(n)`, `choose(a, b, ...)`.

Strumenti ed effetti

Scegliere lo strumento

`:s(x)` imposta lo strumento. Può essere un nome del bank della cartuccia, un preset come `"kick"` o `"epiano"`, oppure un numero. `instruments()` elenca i preset. **[DA COMPLETARE: l'elenco dei nomi degli strumenti pronti, con l'output di instruments()]**

- Uno strumento senza nome è `"triangle"`.
- La batteria senza nota suona alla propria nota (kick 36, snare 50, hat 72); il resto suona a C3.

Il suono di ogni nota

Controllo	Che cosa fa
`:s(x)`	lo strumento (anche un pattern: `:s("<pluck epiano>")`)
`:note(x)`, `:n(x)`	nota, grado
`:gain(x)`, `:velocity(x)`	il volume (0..1)
`:legato(x)`, `:clip(x)`	quanto dura la nota rispetto al suo spazio (1 = tutto)
`:lpf(hz)`, `:hpf(hz)`, `:bpf(hz)`, `:cutoff(hz)`, `:res(x)`	il filtro (passa-basso, passa-alto, passa-banda) e la risonanza 0..1
`:pan(x)`	-1 sinistra, 1 destra
`:room(x)`, `:delay(x)`	quanto va alla stanza e all'eco (`reverb()`, `echo()` li impostano)
`:attack(ms)`, `:decay(ms)`, `:sustain(x)`, `:release(ms)`	l'inviluppo
`:shape(x)`, `:drive(x)`, `:noise(x)`	saturazione, rumore
`:fenv(octaves)`, `:fdecay(ms)`, `:lfo(hz)`, `:wah(octaves)`, `:pwm(x)`, `:duty(x)`	inviluppo del filtro, oscillazione lenta
`:wave(name)`	la forma d'onda (`"saw"`, `"fm"`, `"pluck"`)
`:vib(cent)`, `:vibhz(hz)`, `:detune(cent)`, `:pitch(semitones)`, `:ptime(ms)`	vibrato e inviluppo dell'intonazione
`:fm(x)`, `:fmh(x)`	profondità e rapporto FM
`:raw(true)`	la voce a 8 bit
`:crush(bits)`, `:coarse(n)`	bit crush (restano 1..15 bit), la frequenza divisa (1..16)
`:vowel("a")`	i formanti di una vocale: `"a"`, `"e"`, `"i"`, `"o"`, `"u"`
`:chorus(x)`, `:trem(x)`, `:duck(x)`	l'invio al chorus, il tremolo (su `:lfo`), l'abbassamento delle altre voci (0..1)
`:begin(x)`	dove parte un campione (0..1 della sua lunghezza)

Controllo	Che cosa fa
`:tone{...}`	qualunque chiave di `tone()` : `:tone{curve = "fold", color = "pink", density = .2, reverse = true}`

ATTENZIONE

Le unità sono quelle di `tone()` : millisecondi, pan da -1 a 1, risonanza da 0 a 1. Non sono quelle di Strudel, dove l'attacco è in secondi e il pan va da 0 a 1.

Il kit

`s "kit:2"` (o `s "kit" :n "<0 2>"`) sceglie un colpo del kit della console, suonato alla sua velocità (C4 se non dai una nota). `pump` , il kick del kit che abbassa il resto, funziona allo stesso modo.

- I colpi per nome, come in Strudel: `s "bd sd hh oh cp cb"` . `sd:2` è due semitoni sopra, come per ogni colpo di batteria.
- `rim` e `tom` sono preset del sintetizzatore. Quelli del kit sono `kit:5` e `kit:6` .
- `R.bake` tiene `kit:0` , `kit:1` come strumenti a sé.

DA COPIARE

```
beat = s "bd [~ bd] ~ bd, ~ cp, hh*8?" :crush(8)
pad  = chord "<Am F>" :s "choir" :vowel("<a o>") :chorus(.5) :trem(.3)
```

Pattern, canzoni e struttura

Più parti insieme

In RIFF una canzone è un insieme di **pattern con nome** che suonano insieme. Ogni nome è una parte: drums, bass, keys.

- Con `R.play(name, p)` dai un nome a un pattern. Suonare con un nome già esistente sostituisce ciò che c'era.
- Con `R.code`, ogni globale a cui dai un pattern suona con quel nome (`drums = s "kick*4"`).
- `stack(a, b, ...)` mette più pattern insieme in uno.

Strofa e ritornello

`arrange({4, a}, {4, b})` mette una dopo l'altra sezioni di più cicli: qui quattro cicli di `a`, poi quattro di `b`.

`<a b c>` cambia a ogni ciclo, `cat(a, b, ...)` e `timecat({3, a}, {1, b})` fanno lo stesso su pattern interi.

DA COPIARE

```
local verse  = chord "<Am F C G>" :s "pad"
local bridge = chord "<F C G Am>" :s "pad"
song = arrange({4, verse}, {4, bridge})
```

Ripetere e cambiare

`:every(4, fast(2))` cambia un ciclo su quattro. `:sometimes(add(12))` alza a caso alcune note di un'ottava. Con questi si fa una canzone che si ripete ma non suona mai uguale.

Tempo

Chiamata	Che cosa fa
`setcps(x)`	cicli al secondo (0.5 all'inizio)
`setcpm(x)`	cicli al minuto: `setcpm(30)` sono 30 cicli al minuto
`bpm(x, [beats])`	movimenti al minuto, quattro per ciclo: `bpm(120)`

La velocità cambia senza saltare un movimento. In `R.code` scrivile senza `R.`; in un gioco, `R.setcpm(30)`.

Trasformare un pattern in una canzone del bank

In **bm Sound**, **Ctrl+Enter** sulla riga di F7 mette 1, 2, 4, 8 o 16 battute (Tab) nel bank come nuova canzone con i suoi pattern, al tempo del riff (16 passi per ciclo).

- Gli strumenti che mancano al bank diventano suoni del bank.
- Una nota per passo e per traccia, fino a 8 tracce. Filtro, posizione e stanza di ogni nota restano quelli dello strumento.

Usare una canzone RIFF in un gioco

Le tre chiamate che servono a ogni gioco

```
local R = require "riff"

function _init()
  R.setcpm(30)
  R.play("drums", R.s "kick*4, ~ snare, hat*8")
  R.play("bass", R.note "<c2 a1 f1 g1>" :s "acid" :lpf(R.sine:range(300, 1800):slow(4)))
end

function _update()
  R.update()                      -- ogni frame: le note dei momenti successivi in coda
end
```

1. `local R = require "riff"` in cima a `main.lua`.
2. `R.play(name, pattern)` una volta, in `_init` o quando cambia la scena.
3. `R.update()` in `_update`, **a ogni frame**. Senza, non suona niente.

Che cosa fa un frame lento

Un frame lento non perde il tempo. Dopo una pausa (il gioco sospeso) le note già passate vengono saltate.

Le funzioni per il gioco

Funzione	Che cosa fa
`R.play(name, p)`, `p:play(name)`	suona p con quel nome (sostituendo ciò che c'era)
`R.stop(name)`, `R.hush()`	ferma uno, tutti
`R.update()`	da chiamare in `_update`: mette in coda le note dei momenti successivi
`R.setcps(x)`, `R.setcpm(x)`, `R.bpm(x, [beats])`	la velocità (cambia senza salti)
`R.playing()`, `R.get([name])`	i nomi che suonano; il pattern di un nome (senza nome: tutti insieme)
`R.now()`	il ciclo di adesso
`R.voices(v, ...)`	le voci che può usare (nessuna indicata: tutte e 8)
`R.errors()`	gli errori dei pattern (un pattern con un errore si ferma)
`R.code(text)`	codice dal vivo
`R.bake(p, {cycles=, steps=16})`	il pattern come brano per il bank (nella forma di `ai.music`)
`R.piece(t)`	un brano di `ai.music` come pattern

Codice dal vivo dal gioco

`R.code(text)` esegue un pezzo di Lua dove le funzioni di RIFF sono globali (`note` , `s` , `stack` , `sine` , `setcpm` , `hush`) e poi quelle del gioco.

- **Ogni globale a cui dai un pattern suona con quel nome.**
- Un nome che inizia con `_` non suona: `_drums = ...` lo mette in silenzio.
- I pattern di un `R.code` precedente che non compaiono più si fermano.
- Un `local` è solo un pezzo.
- Restituisce `true` , oppure `nil` e l'errore. Quando c'è un errore, la musica precedente continua.
- Prima di suonare prova ogni pattern sul suo primo ciclo: un errore (una nota che non esiste, una scala sbagliata) salta fuori subito.

Voci, effetti sonori e canzoni del bank

RIFF usa le voci del sintetizzatore. La voce si sceglie quando la nota parte: una libera, la coda di un suono, o la nota più vecchia di RIFF. Non prende mai le voci di una canzone del bank né di un effetto sonoro. `R.voices(4, 5, 6, 7)` limita le voci che può usare.

La chiamata di basso livello che c'è sotto è `play_at(t, sound, [note], [ms], [vol], [tag])` , con l'orologio del suono `audio_time()` ; `play_cancel([tag])` dimentica le note in attesa. RIFF le usa per te.

PROVA

Fai un gioco con il codice qui sopra ed eseguilo. Senti la batteria e il basso. Poi chiama `R.hush()` dal tuo codice: la musica si ferma. **[DA COMPLETARE: un esempio provato di evento del gioco (un pulsante, un cambio di livello) che chiama R.stop o R.play]**

Esempi

Tutti questi girano nei test (`make test-riff`).

Un ritmo, un basso e le tastiere

DA COPIARE

```
setcpm(30)
drums = s "kick*4, ~ snare, hat*8" :gain(.9)
bass  = note "<c2 a1 f1 g1>" :s "acid" :lpf(sine:range(300, 1800):slow(4))
keys  = chord "<Cm7 Ab Eb Bb>" :s "epiano" :arp("updown") :fast(2) :room(.4)
```

Un pad, un arpeggio e la batteria

DA COPIARE

```
setcpm(28)
local verse  = chord "<Am F C G>"
pad   = verse :s "pad" :room(.6) :gain(.7)
arp   = verse :s "pluck" :arp("updown") :fast(4) :pan(sine:range(-.5, .5))
drums = s "kick ~ ~ kick, ~ snare, hat*8?" :every(4, fast(2))
```

Una melodia su una scala

DA COPIARE

```
setcpm(30)
drums = s "kick*4, ~ snare, hat*8"
bass  = note "<c2 a1 f1 g1>*2" :s "acid" :lpf(800)
lead  = n "0 2 4 <7 6>" :scale("A:minor") :s "pluck" :sometimes(add(12))
```

Il kit e gli effetti

DA COPIARE

```
setcpm(30)
drums = s "bd sd [~ bd] sd, hh*8, kit:3(3,8)" :crush(8)
pad   = chord "<Am F>" :s "choir" :vowel("<a o>") :chorus(.5) :trem(.3)
bass  = note "<a1 f1>*4" :s "bitbass" :coarse(4)
```

Righe singole per la riga di F7

DA COPIARE

```
s "kick ~ ~ kick, ~ snare, [~ hat]*4" :swing(4)
note "<c2 c2 eb2 g1>*4" :s "acid" :lpf(sine:range(300, 1800):slow(4))
chord "<Am F C G>" :s "epiano" :arp("updown") :fast(4)
n "0 2 4 <7 6> 4 2 1 ~" :scale("A:minor") :s "pluck"
s "kick(3,8), hat(7,16), ~ clap"
note "c4 e4 g4 c5" :s "chip" :every(4, rev) :fast(2)
```

Fermare e far partire dal codice

DA COPIARE

```
R.play("theme", R.note("c4 e4 g4 c5"):s("epiano"):room(0.4))
R.play("beat", R.s("kick ~ kick snare"))
R.hush()
```

Errori comuni

Che cosa vedi	Perché	Che cosa fare
Non suona niente in un gioco	`R.update()` non viene chiamato	chiamalo in `_update`, a ogni frame
`riff: missing ] at 6 in "a [b c"`	una parentesi non è chiusa	chiudila; il messaggio dice dove
Un errore con una parola di nota	la parola non è una nota, per esempio `note "zz"`	usa nomi come `c`, `eb4`, `f#2` oppure numeri
Un pattern si ferma da solo	uno strumento sconosciuto, per esempio `s "nothing*4"`: il pattern si ferma quando suona	leggi `R.errors()`, usa un nome da `instruments()` o dal bank
Una riga di `R.code` è muta	il nome inizia con `_`, oppure è un `local`	dalle un nome globale senza `_`
La vecchia musica continua dopo un refuso	`R.code` restituisce `nil` e l'errore, e tiene la musica precedente	correggi l'errore ed eseguilo di nuovo
Una parte tolta dal codice si ferma	i pattern del `R.code` precedente che non compaiono più si fermano	scrivi in ogni `R.code` tutte le parti che vuoi
Attacco o pan suonano male	le unità sono quelle di `tone()`: millisecondi, pan -1..1	non usare i secondi di Strudel né il pan 0..1
Un colpo di batteria ha l'altezza sbagliata	la batteria senza nota usa la propria nota	usa `kick:2` per salire di due semitoni

Punti aperti

- **La tua prima canzone in 5 minuti**: come avviare bm Sound dal menu della console
- **La tua prima canzone in 5 minuti**: come aprire bm Code e una scheda nuova
- **La tua prima canzone in 5 minuti**: immagine da fare: shots/riff-sound-line.png
- **Strumenti ed effetti**: l'elenco dei nomi degli strumenti pronti, con l'output di instruments()
- **Usare una canzone RIFF in un gioco**: un esempio provato di evento del gioco (un pulsante, un cambio di livello) che chiama R.stop o R.play